There are a few key pieces we must make sure are in place before proceeding with the rest of our chatbot project:

- **Source data preparation**: `catalog.database.pdf_documentation_text` is the Delta table you have prepared with your self-managed embedding functions as the primary data repository for their vector search index.
- **Vector search index creation**: An index named `catalog.database.docs_vsc_idx_cont` using Databricks VS. This index is linked to the source Delta table. The index is set on a trigger base update (`pipeline_type="TRIGGERED"`), which means that any modifications or new additions to the historical texts should be synced manually to your vector search index. If you wish to have changes reflected in the vector search index automatically, choose the `CONTINUOUS` mode instead. This continuous update mechanism ensures the data is always current and ready for analysis.
- **Embedding model integration**: To transform the text data into a vector format, we are using self-managed embeddings. This means no model for embedding conversion is provided. However, you may want to choose managed embeddings, where `text_chunks` is automatically converted into embeddings with a model that is served through Databricks Model Serving. This can be specified in the setup under the `embedding_model_endpoint_name` parameter. This integration guarantees that the textual data is efficiently converted into vectors, making it suitable for advanced similarity searches.

We import the `wait_for_index_to_be_ready()` function from the `utils` folder. We run the code in *Figure 4.28* to repeatedly check the index's status until it is in an "online" state. As the VS index only needs to be created once, this function can take some time before the embeddings reach your VS index. Proceed once your index is in a "ready" state.

```
wait_for_index_to_be_ready(vsc, vsc_endpoint_name, vs_index_fullname)
```

Figure 4.28 – The wait for index function checks for the index status until it is online and ready

Once the index is online, we call `get_index()` to perform a similarity search. We also call a `describe()` method to show you the broader API options you have.